PROJECT REPORT / PRESENTATION: DELIVERABLE 1

Distributed Task Notification and Worker System

Course: Computer Networks / SDN Mini-Project

1. Problem Understanding & Architecture

- **Project Objective:** To design, implement, and evaluate a distributed task notification and worker system using low-level TCP socket programming integrated within a Mininet-based network environment.
- **System Architecture Overview:** The system follows a centralized broker-worker model operating over a virtual network. It consists of three primary software components:
    - **topo.py (Custom Topology):** Defines the virtual network topology containing a central broker host (10.0.0.10), multiple worker hosts (w1 at 10.0.0.1, w2 at
      10.0.0.2), and an Open vSwitch switch connecting them.
    - **broker.py (Central Server):** Listens for incoming TCP connections, manages worker registrations, tracks active connections, and dispatches tasks. ○ **worker.py (Client Node):** Connects to the broker, registers its identity, listens for assigned tasks, executes payloads, and reports results back.
- **Communication Flow:** Nodes communicate via reliable TCP socket streams using an application-level JSON protocol delimited by newline characters (\n).

2. TCP Socket Implementation

- **Socket Programming:** Implemented using Python's low-level socket library (socket.AF_INET, socket.SOCK_STREAM) to establish reliable point-to-point connections over TCP.
- **Concurrency:** The broker utilizes Python's threading module (threading.Thread) to handle multiple concurrent worker connections simultaneously without blocking.
- **Protocol Message Exchange:**
    - REGISTER: Sent from a worker to the broker upon initial connection ({"type": "REGISTER", "worker_id": "worker_1"}).

○ TASK: Dispatched from the broker to assign a workload ({"type": "TASK", "task_id": "...", "payload": "..."}). ○ RESULT: Sent back from the worker to report completion ({"type": "RESULT", "task_id": "...", "result": "..."}).

- **Error Handling:** Graceful exception handling and cleanup routines ensure disconnected sockets are safely closed and resources are reclaimed.

3. Mininet & SDN Network Implementation

- **Environment:** Deployed and tested inside an Ubuntu virtual machine running Mininet and Open vSwitch (OVS).
- **Topology Configuration (topo.py):** Configured with an OVS switch operating in standalone mode (failMode='standalone') to ensure seamless local packet forwarding and namespace isolation.
- **Socket-SDN Integration:** End-to-end integration is verified by launching application processes inside distinct Mininet host namespaces (broker, w1, w2), proving that the socket application successfully routes over the virtualized data plane.

4. Initial Testing & Demonstration Results

- **Test Case 1: Single Worker Connection & Registration** ○ *Action:* Start broker.py in the background and launch w1.
  - *Result:* Successful TCP handshake, connection confirmation, and registration response ({'status': 'REGISTERED'}).
- **Test Case 2: Multi-Worker Concurrency & Task Dispatching** ○ *Action:* Launch both w1 and w2 concurrently alongside the broker.
  - *Result:* The broker independently handles multiple active worker threads, automatically dispatches tasks, and logs successful result reporting from each worker in parallel.

Source Codes:

- broker.py

```
1. import socket
2. import threading
3. import json
4. import time
5.
6. HOST = '0.0.0.0'
```

```
7. PORT = 5000
```

```
8.
9. active_workers = {}
10.
11.    def handle_worker(conn, addr):
12.    print(f"\n[+] Connected to worker at {addr}") 13.
worker_id = None
14.    try:
15.    # 1. Receive registration 16.         data =
conn.recv(1024)
17.             if not data:
18.             return
19.             message = json.loads(data.decode('utf-8'))
20.             if message.get("type") == "REGISTER":
21.             worker_id = message.get("worker_id")
22.             active_workers[worker_id] = conn
23.             print(f"[REGISTRATION] Worker registered: {worker_id}")
24.
25.             # Send welcome response
26.             response = {"status": "REGISTERED", "message": "Welcome to
                the broker"}
27.             conn.sendall((json.dumps(response) + "\n").encode('utf-8'))
28.
29.    # 2. Automatically send a task after 3 seconds
30.    time.sleep(3)
31.    if worker_id in active_workers:
32.    task_msg = {"type": "TASK", "task_id": "task_101",
   "payload": "Process Data Batch A"}
33.    conn.sendall((json.dumps(task_msg) + "\n").encode('utf-8')) 34.
print(f"[*] Automatically sent task_101 to {worker_id}")
35.
36.                 # 3. Listen for results
37.                 while True:
38.                 data = conn.recv(1024)
39.                 if not data:
40.                 break
41.                 res_message = json.loads(data.decode('utf-8'))
42.                 if res_message.get("type") == "RESULT":
43.                 print(f"\n[RESULT] Task {res_message.get('task_id')}
                    completed by {worker_id}: {res_message.get('result')}")
44.
45.             except Exception as e:
46.             print(f"[-] Error with worker {worker_id}: {e}")
47.             finally:
48.             if worker_id and worker_id in active_workers:
49.             del active_workers[worker_id]
50.             conn.close()
51.             print(f"[-] Connection closed for {worker_id}")
52.
```

```
53.     def start_broker():
54.     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
55.     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
56.     server.bind((HOST, PORT))
57.     server.listen(5)
58.     print(f"[*] Broker listening on {HOST}:{PORT}")
59.
60.             try:
61.             while True:
62.             conn, addr = server.accept()
63.             threading.Thread(target=handle_worker, args=(conn, addr),
                daemon=True).start()
64.             except KeyboardInterrupt:
65.             print("\n[*] Stopping Broker...")
66.             finally:
67.             server.close()
68.
69. if __name__ == "__main__": 70.
start_broker()
71.
```

- worker.py

```
1.     import socket
2.     import json
3.     import time 4. import sys
5.
6.  BROKER_IP = sys.argv[2] if len(sys.argv) > 2 else '10.0.0.10'
7.  PORT = 5000
8.  WORKER_ID = sys.argv[1] if len(sys.argv) > 1 else "worker_1"
9.
10.     def start_worker():
11.     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12.     try:
13.     print(f"[*] Attempting to connect to broker at
        {BROKER_IP}:{PORT}...")
14.     client.connect((BROKER_IP, PORT)) 15.         print(f"[*] Connected
        to broker!")
16.
17.    # Register
18.    reg_msg = {"type": "REGISTER", "worker_id": WORKER_ID} 19.
client.sendall((json.dumps(reg_msg) + "\n").encode('utf-8'))
20.
21.    data = client.recv(1024)
22.    response = json.loads(data.decode('utf-8')) 23.
print(f"[Server Response]: {response}")
24.
25.         # Listen for tasks from broker
```

```
                while True:
                data = client.recv(1024)
                if not data:
                break
                message = json.loads(data.decode('utf-8'))
                if message.get("type") == "TASK":
                task_id = message.get("task_id")
                payload = message.get("payload")
                print(f"\n[RECEIVED TASK] {task_id} with payload:
                '{payload}'. Processing...")

                 # Simulate work execution time
                 time.sleep(2)

                # Send result back
                result_msg = {"type": "RESULT", "task_id": task_id,
   "result": "SUCCESS - Processed payload"}
                client.sendall((json.dumps(result_msg) +
                "\n").encode('utf-8'))
                print(f"[SENT RESULT] Task {task_id} completed and
                reported to broker.")

        except ConnectionRefusedError:
        print("[-] Connection failed. Is the broker running?")
        except KeyboardInterrupt:
        print("\n[*] Disconnecting worker...")
        finally:
        client.close()

if __name__ == "__main__":
    start_worker()

```

- topo.py

```
from mininet.net import Mininet
from mininet.node import OVSKernelSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info

    def run_topology():
    # Create network without a controller, using standalone switch mode
    for native learning
    net = Mininet(switch=OVSKernelSwitch, autoSetMacs=True)

    info("*** Adding OpenFlow Switch (Standalone Mode)\n")
    s1 = net.addSwitch('s1', failMode='standalone')

    info("*** Adding Hosts (Broker and Workers)\n")
```

```
14.     broker = net.addHost('broker', ip='10.0.0.10')
15.     w1 = net.addHost('w1', ip='10.0.0.1')
16.     w2 = net.addHost('w2', ip='10.0.0.2')
17.
18.     info("*** Creating Links\n")
19.     net.addLink(broker, s1)
20.     net.addLink(w1, s1)
21.     net.addLink(w2, s1)
22.
23.     info("*** Starting Network\n") 24.
net.start()
25.
26.     info("\n*** Network is live! Use the mininet prompt below:\n") 27.
CLI(net)
28.
29.     net.stop()
30.
31.    if __name__ == '__main__':
32.    setLogLevel('info') 33.     run_topology()
34.
```

- **Execution commands**:

1. Run the custom Mininet topology script:
sudo python3 topo.py

2. Inside the Mininet CLI prompt (mininet>), start the broker in the background:
broker python3 broker.py &

3. Start the worker nodes to connect, register, and process tasks concurrently:
w1 python3 worker.py worker_1 10.0.0.10 &
w2 python3 worker.py worker_2 10.0.0.10

4. Exit:- (cntrl+c) , type exit